

Chat With This Page — Project Overview

A Chrome extension that lets a user chat with the content of their currently open webpage, backed by a multi-provider LLM proxy server, deployed with a Docker + CI/CD + monitoring pipeline. Framed as an **LLMOps** project (operating an LLM-powered app in production), not classic MLOps (no model training/fine-tuning involved).

1. What it does

1. User clicks the extension icon on any webpage.
 2. The extension extracts the page's text (and optionally a screenshot for vision-based questions).
 3. The user asks a question in a popup chat window.
 4. The question + page content is sent to a backend proxy server.
 5. The proxy forwards the request to the chosen LLM provider (OpenAI, Groq, Anthropic, or a local model) and streams the answer back.
 6. The answer renders in the popup, with optional features like quote-jumping, summarization, and per-tab history.
-

2. Architecture

```
[Popup UI] --(extract page text)--> [Content script]
[Popup UI] --(question + context)--> [Background service worker]
[Background service worker] --(HTTPS request)--> [FastAPI proxy server on EC2]
[FastAPI proxy] --(forwards request)--> [LLM provider: OpenAI / Groq / Anthropic]
[FastAPI proxy] <--(streamed response)-- [LLM provider]
[Background service worker] <--(streamed response)-- [FastAPI proxy]
[Popup UI] <--(rendered answer)-- [Background service worker]
```

Why a backend proxy at all (vs. calling the LLM directly from the extension):

- Keeps the API key off the client, out of the extension's source code
- Central place to add logging, monitoring, rate limiting, and provider fallback
- Lets the same backend later serve other clients (mobile, web app) if needed

3. Components

3.1 Chrome Extension (Manifest V3)

File	Purpose
<code>manifest.json</code>	Declares permissions, popup, background worker, icons
<code>popup.html</code> / <code>popup.js</code> / <code>popup.css</code>	The chat window UI
<code>content.js</code>	Injected into the active tab to extract page text/DOM
<code>background.js</code>	Service worker — builds the prompt, calls the backend, relays streamed response
<code>options.html</code> / <code>options.js</code>	Settings page — provider selection, model, API key (if not using the backend's stored key)

Manifest V3 notes:

- Uses `activeTab` + `scripting` permissions, not blanket host access — only gets page access when the user clicks the icon
- The background script is a **service worker**, not a persistent page — it goes idle after ~30s of inactivity and is woken by events. Any state that needs to survive must go in `chrome.storage`, not a JS variable.
- CSP in MV3 blocks remote code execution (`eval()`, remote `<script src>`) — all libraries must be bundled locally in the extension.

3.2 Backend — FastAPI proxy

```
main.py           # FastAPI app: /chat (streaming) and /health endpoints
requirements.txt  # fastapi, uvicorn, httpx
Dockerfile        # containerizes the service
.env              # API keys (not committed to git)
```

Core endpoint logic:

- `POST /chat` — accepts `{ provider, model, messages, stream }`, looks up the right provider config, forwards the request with the server-side API key, streams the response back chunk by chunk

- `GET /health` — used by Docker healthchecks and uptime monitoring

Provider adapter pattern — each provider (OpenAI, Groq, Anthropic, local Ollama) is a small config entry (`url`, `key_env`, request shape), so adding a new provider is a few lines, not a rewrite.

3.3 Infrastructure

Layer	Tool	Notes
Containerization	Docker	One image, runs identically locally and on EC2
Compute	AWS EC2 (t2/t3.micro)	Free tier for 12 months, then a small ongoing cost
CI/CD	GitHub Actions	On push to <code>main</code> : build image → push to Docker Hub → SSH into EC2 → pull + redeploy
Secrets	GitHub Actions secrets (CI) + <code>.env</code> on EC2	API keys never committed to the repo
Reverse proxy / HTTPS	Nginx + Let's Encrypt (Certbot)	Required since the extension calls the backend over HTTPS
Uptime monitoring	Uptime Kuma (self-hosted, free)	Pings <code>/health</code> , dashboard + alerting (email/Discord)
Application monitoring	Python <code>logging</code> → AWS CloudWatch Logs	Per-request latency, provider/model used, status, errors

4. Feature list

Core (v1)

- Extract and chat with current page text
- Streaming responses
- Settings page: provider, API key, model
- Per-tab conversation memory
- Basic error handling

Multi-LLM support

- Provider switching (OpenAI, Groq, Anthropic, local Ollama)
- Per-question model override
- Automatic fallback to a backup provider on failure
- Cost/speed indicator per model
- Multiple stored API keys

Page understanding

- "What's on screen" mode (screenshot + vision model)
- Selected-text chat (right-click context menu)
- Quote-and-jump to cited page sections
- Chunking/summarization for long pages
- Semantic search over page content
- Table extraction for data questions
- PDF support (pdf.js)
- YouTube transcript chat
- Code block detection/explanation

Multi-tab / research

- Chat across multiple open tabs
- Compare two page versions
- Cross-tab semantic search
- Running research notebook

Quick actions

- One-click Summarize / TL;DR / Translate / Extract action items / Simplify
- Suggested follow-up question chips
- Export chat as markdown
- Save answers to external notes apps

Personalization

- Persistent per-site memory
- Custom system prompt/persona
- Searchable cross-page history
- Dark mode

Trust & privacy

- "Context sent to provider" transparency panel
- Local-only mode via Ollama
- In-extension privacy disclosure
- `activeTab`-only permissions

Advanced / later

- Light agentic actions (e.g. autofill), gated behind explicit confirmation
- Keyboard shortcut to open popup
- Live token/cost estimate

5. LLMOps layer (what makes this more than a deployed API)

Practice	How it's applied here
Reproducibility	Docker image — same environment locally, in CI, and on EC2
Automated deploys	GitHub Actions — push to <code>main</code> triggers build + redeploy, no manual SSH
Observability	Structured request logs (provider, model, latency, status) shipped to CloudWatch
Uptime monitoring	Uptime Kuma dashboard + alerts on <code>/health</code>
Evaluation (future step)	Small test set of (page, question, expected answer quality) run against each provider/model to catch regressions when prompts or providers change
Secrets management	API keys never in source control; CI secrets + server-side <code>.env</code>

6. Build order

1. Build extension core: popup, content script, page extraction
2. Build FastAPI backend with a single provider (e.g. Groq) working end-to-end
3. Add Docker, confirm the container runs locally

4. Deploy manually to EC2 once (sanity check), add Nginx + HTTPS
 5. Add GitHub Actions for automated build + deploy
 6. Add Uptime Kuma for uptime monitoring
 7. Add structured logging → CloudWatch for per-request visibility
 8. Layer in extension features (multi-LLM switching, quick actions, semantic search, etc.)
 9. (Optional, for full LLMOps story) Add a small evaluation script comparing providers/prompts
-

7. Deployment options recap

Path	Use case
Load unpacked (<code>chrome://extensions</code> → Developer Mode)	Personal use — no deployment needed at all
Chrome Web Store	Public release — \$5 one-time fee, requires privacy policy, review process
EC2 + Docker + GitHub Actions	This project's chosen path — gives a real backend to monitor and operate
Cloudflare Workers / Render / Fly.io	Lower-maintenance alternative to EC2 if server upkeep becomes a hassle

8. Open questions to resolve while building

- Which provider(s) to support first (recommend starting with just Groq — fast and has a free tier)
- Whether to keep API keys server-side only, or also allow users to bring their own key from the extension settings
- How much page content to send by default vs. when to trigger chunking/summarization
- Whether evaluation is worth building out fully, or kept lightweight as a manual spot-check script